Lecture-6

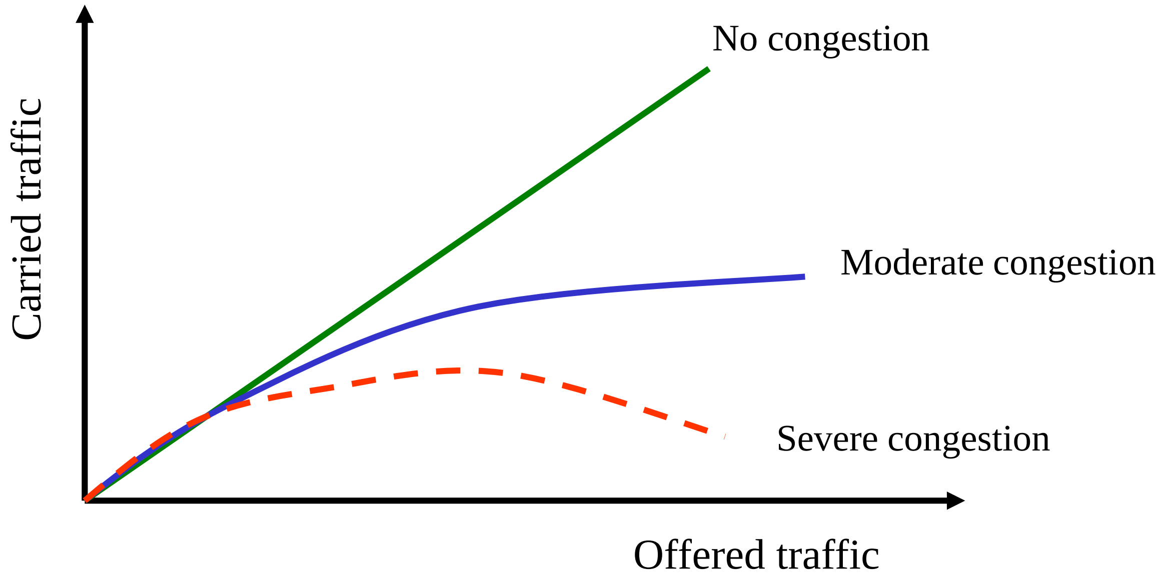
Congestion Control and QoS

Dr. Md. Imdadul Islam
Professor, Department of Computer Science and Engineering
Jahangirnagar University
https://www.juniv.edu/teachers/imdad

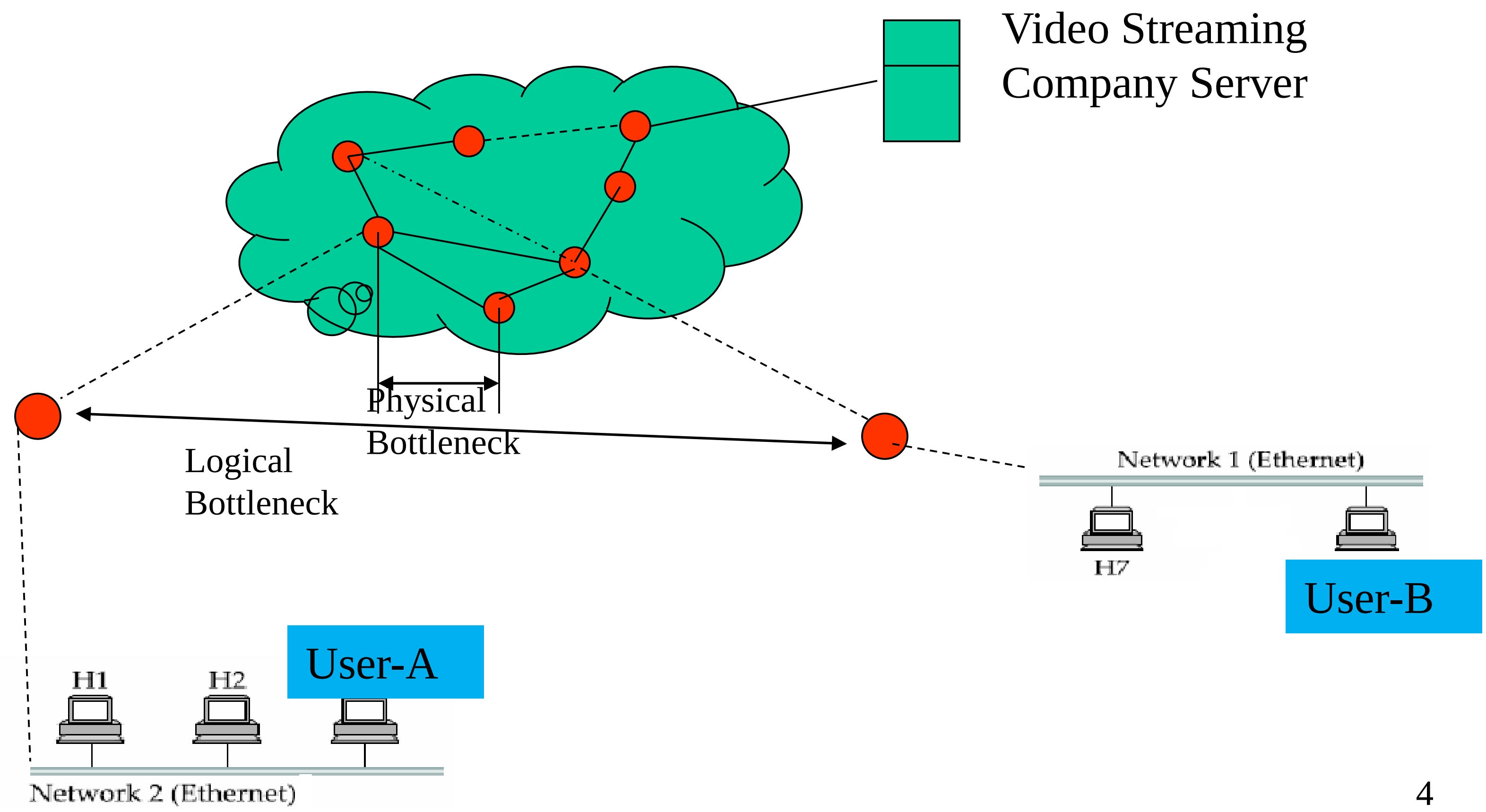
Congestion control at Network layer

- ✓ **Congestion** (overloaded condition in a region of a network) may occur if the number of datagrams sent by source computers is beyond the capacity of the network or routers. In this situation, some routers may drop some of the datagrams.
- ✓ Too many packets present in the network causes **packet delay** and **loss** that degrades performance.
- ✓ If the congestion continues, sometimes a situation may reach a point where the system collapses and no datagrams are delivered.

- ✓ Fig below shows a set of performance graphs for networks in which three possible cases are compared: no congestion, moderate congestion and severe congestion.
- ✓ Congestion control can be achieved by optimum usage of the available resources (buffer of router and link capacity) in the network.



Congestion can be either logical or physical, as shown in fig. below. The queuing feature (queueing's delay) in two routers can create a **logical bottleneck** between user A and user B. Again, insufficient capacity BW on physical link between routers and the network can also be a bottleneck, resulting in **physical congestion**.



Resource shortage can occur:

✓At the data link layer, where the link BW runs out.

✓At the network layer, where the queues of packets at nodes go out of control

✓At the transport layer, where logical links between two ends within a communication session go out of control.

One key way to avoid congestion is to carefully allocate network resources (such as BW and buffer space) to users and applications.

Flow control

✓**Flow control**, relates to the point-to-point traffic between a given sender and receiver (not overloading a region of a network like congestion case). Its job is to make sure that a fast sender cannot continually transmit data faster than the receiver is able to absorb it.

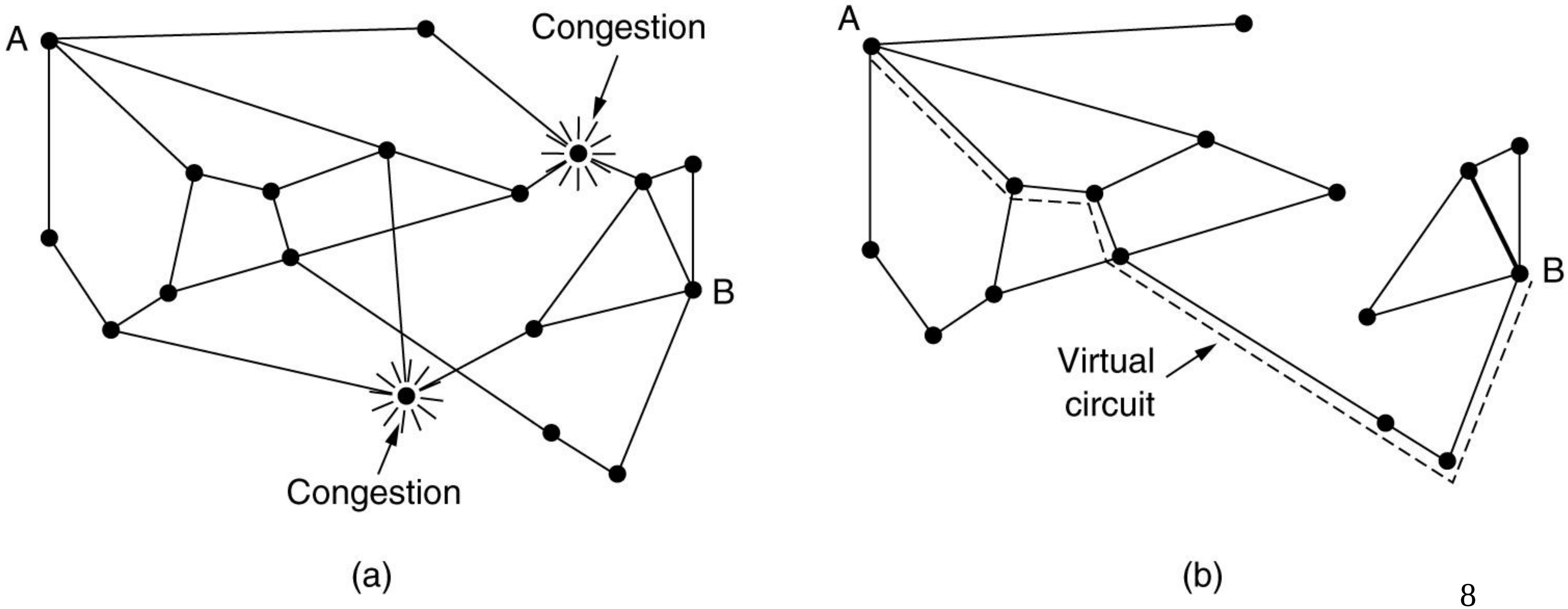
✓Flow control frequently involves some direct feedback from the receiver (for example in stop-and-wait and sliding window) to the sender to tell the sender how things are doing at the other end. Datalink layer deals with flow control.

Congestion control in **virtual circuit** subnets

Admission Control

- ✓ Idea is simple: once congestion has been signaled, no more virtual circuits are set up until the problem has gone away. Thus attempts to set up new transport layer connections fail. While this approach is crude, it is simple and easy to carry out.
- ✓ In telephone system, when a switch gets overloaded, it also practices admission control by not giving dial tones.

An alternative approach is to allow new virtual circuits but carefully route all new virtual circuits far way from the congested router. For example, consider the subnet of fig. (a) below in which two routers are congested. Suppose that a host attached to router A wants to set up a connection to a host attached to router B. Normally this connection would pass through one of the congested routers. To avoid this situation, we an redraw the subnet as shown in fig. (b), omitting the congested routers and all of their lines. The dashed line shows a possible route for the virtual circuit that avoids the congested routers.



✓Another strategy relating to virtual circuits is to negotiate an agreement between the host and subnet when virtual circuit is set up. This agreement normally specifies the volume of and shape of the traffic, **QoS** required and other parameters.

Congestion control in **datagram subnets**

There are several approaches can be applied in datagram subnet to control traffic congestion. In this section we will consider only three techniques:

Load shedding

Load shedding is a fancy way of saying that when routers are being inundated by packets that they can not handle, they just threw them away. The term comes from the worlds of electrical power generation, where it refers to the practice of utilities intentionally blocking out certain areas to save the entire grid from collapsing on hot summer days when the demand of power exceeds supply.

- A router drowning in packets can just pick packets at random to drop, but usually it can do better than that. Which packet to discard may depend on the applications running. **For file transfer**; an old packet is worth more that a new one because dropping packet 6 and keeping packets 7 through 10 will cause a gap at the receiver that may force packets 6 through 10 to be retransmitted. Whereas dropping 10 may require only 10 through 12 to be retransmitted.

- In contrast for multimedia, a new packet is more important that an old one. The former policy (old is better than new) is called **wine** and the latter (new is better than old) is called **milk**.

- For many applications, some packets are more important than others. For example certain algorithms for compressing video periodically transmit an entire frame (I frame) and then send subsequence frames as difference (B or P frame) from the last full frame. In this case dropping a packet that is part of difference is preferable than the full frame.

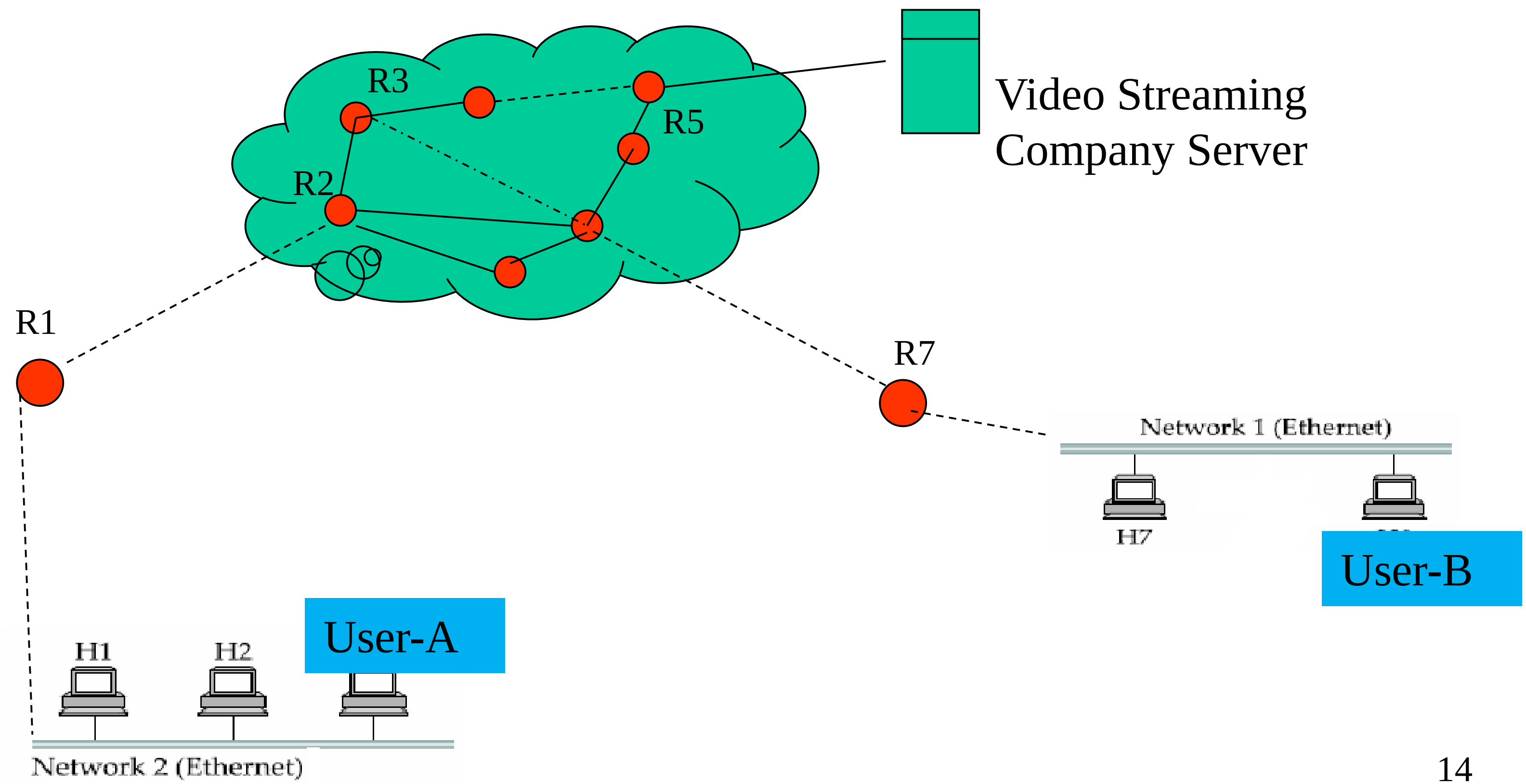
- An another example, consider transmitting a document containing ASCII text and pictures. Losing a line of pixels in some image is far less damaging than losing a line of readable text.

Transmission of choke packets

- ✓ In this scheme **choke** (to block something) **packets** are sent to the source node by a congested node to restricts the flow of packets from the source node.
- ✓ A router or even an end host can send these packets when it is near full capacity, in anticipation of a condition leading to congestion at the router. The choke packets are sent periodically until congestion is relived. On receipt of the choke packets, the source host reduces its traffic generation rate until it stops receiving them.

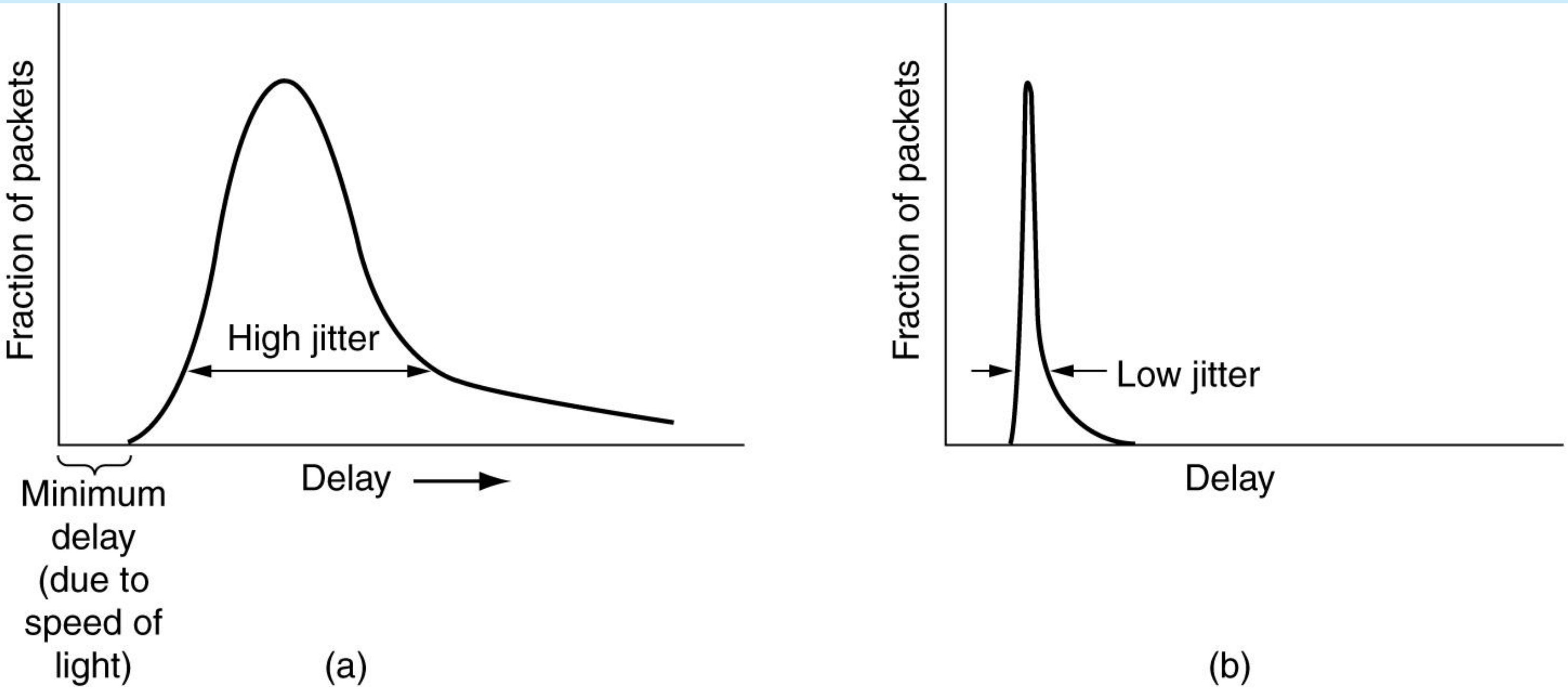
Traffic Policing

- ✓ Monitoring a traffic flow is called **traffic policing**. Traffic policing occurs when a flow of data is regulated so that cells (or frames or packets) that exceed a certain performance/threshold (for example voice calls greater than 1Mbps) level are discarded.
- ✓ An edge router, such a R1 in the fig. acts as a traffic police and directly monitors and controls its immediate connected consumers. In the fig. R1 is policing as a residential dispatch node from which traffic from a certain residential area is flowing into the network.



Jitter Control

For applications such as audio and video streaming, it does not matter much if the packet takes 20 ms or 30 ms to be delivered, as long as the transmit time is constant. The variation (standard deviation) in the packet arrival times is called jitter. High jitter, for example, having some packets taking 20 ms and others taking 30ms to arrive will give an uneven quality of sound or movie. Jitter is illustrated graphically below.



(a) High jitter. (b) Low jitter.

When a packet arrives at a router, the router checks to see how much the packet is behind or ahead of its schedule. This information is stored in the packet and updated in each hop. In the packet is **ahead of schedule**, it is held just long enough to get it back on schedule. If it is **behind schedul**e, the router tries to get it out the door quickly.

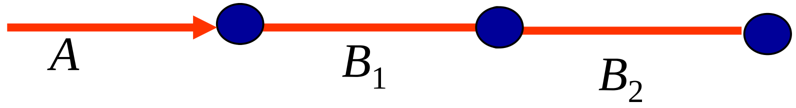
A Quick Estimation of Link Blocking

Point-to-point blocking

- ✓ A telecommunication network/Internet is actually the series parallel combinations of nodes/exchanges/routers connected by links/circuits/channels/trunks.
- ✓ Carried traffic of one link is considered as the offered traffic on the end node.
- ✓ Lost traffic of one node is carried by the parallel alternate channels.
- ✓ The model is applicable for datagram traffic.

Series links

When several links are connected in series, the carried traffic of 1st link becomes the offered traffic on second link and that of second link as the offered traffic of third link and so on. Let us consider two links with blocking probability of B_1 and B_2 are in series like fig. on next slide.



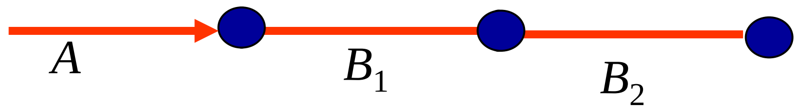


Fig. Two links are in series

Carried and lost traffic of individual combined link is shown in table-1.

Table-1

Link	Offered	Carried	Lost
1	A	$A(1\text{-}B_1)$	AB_1
2	$A(1\text{-}B_1)$	$A(1\text{-}B_1)(1\text{-}B_2)$	$A(1\text{-}B_1)B_2$
Combined	A	$A(1\text{-}B_1)(1\text{-}B_2)$	$AB_1 + A(1\text{-}B_1)B_2$

Overall call blocking probability,

B= overall lost traffic/overall offered traffic

=1- overall carried traffic/overall offered traffic

$=1\text{-} A(1\text{-}B_1)(1\text{-}B_2)/\mathrm{A}=1\text{-} (1\text{-}B_1)(1\text{-}B_2)$

If L links are in series then,

$B=1\text{-}(1\text{-}B_1)(1\text{-}B_2)\ldots(1\text{-}B_L)= 1\text{-}\prod_{i=1}^{L}(1 - B_i)$

Links in parallel

When links are in parallel then lost/overflow traffic of 1st link is carried by the second link and that of 2nd link is carried by the third link and so on. Two parallel links are shown in fig.2.10.

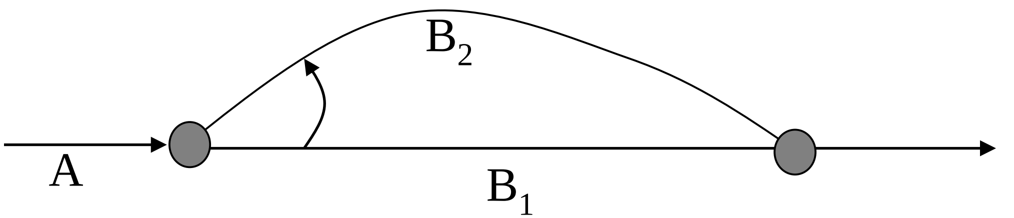


Fig.2.10 Two links are in parallel

Link	Offered	Carried	Lost
1	A	$A(1\text{-}B_1)$	AB_1
2	AB_1	$AB_1(1\text{-}B_2)$	AB_1B_2
Combined	A	$A(1\text{-}B_1)+AB_1(1\text{-}B_2)$	AB_1B_2

Overall call blocking probability,

B=overall lost traffic/overall offered traffic

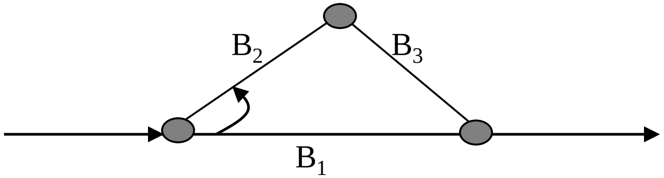
$= AB_1B_2/A = B_1B_2$

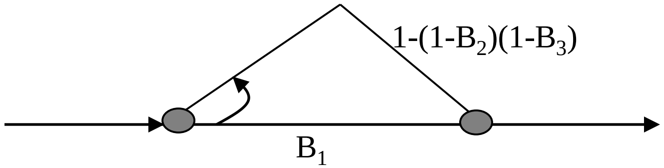
If L links are in parallel then,

$$B = \prod_{i=1}^{L} B_i$$

Example-1

Determine overall blocking probability of the following network.



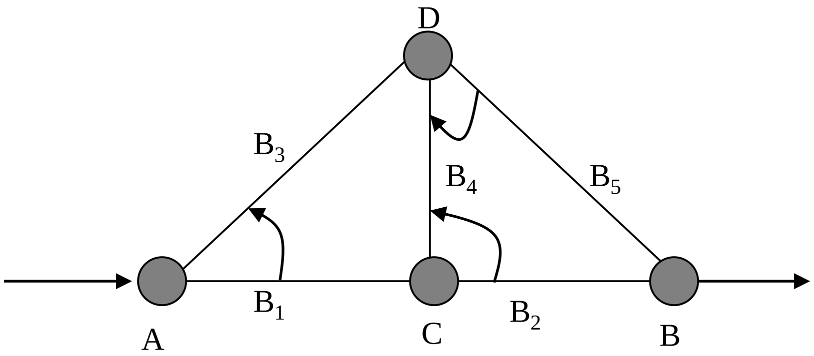


Equivalent circuit

$$B = B_1\{1\text{-}(1\text{-}B_2)(1\text{-}B_3)\}$$

Example-2

Following nodes (fig. 2.14) are the part of a big network. Determine the blocking probability for the available paths:
A-C-B, A-D-B, A-D-C-B and A-C-D-B

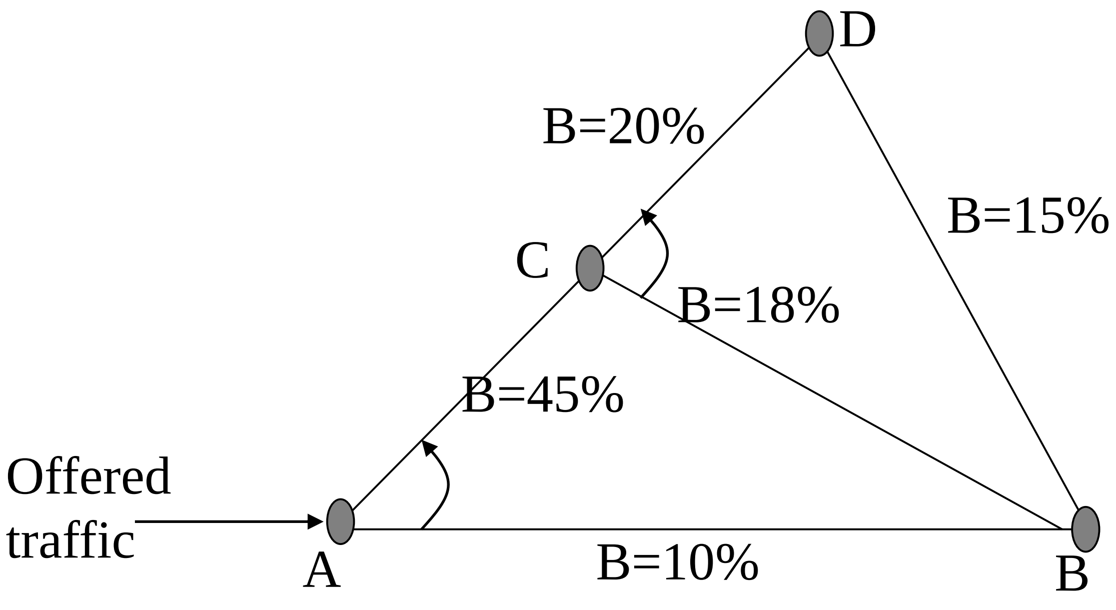


Path	Carried traffic
A-C-B	$A(1-B_1)(1-B_2)$
A-D-B	$AB_1(1-B_3)(1-B_5)$
A-D-C-B	$AB_1(1-B_3)B_5(1-B_4)(1-B_2)$
A-C-D-B	$A(1-B_1)B_2(1-B_4)(1-B_5)$

Let the total carried traffic, $\overline{X}$ then, B=1- $\overline{X}$ /A

Exercise

For the following four nodes network (fig. below) determine overall blocking probability between node A and B.



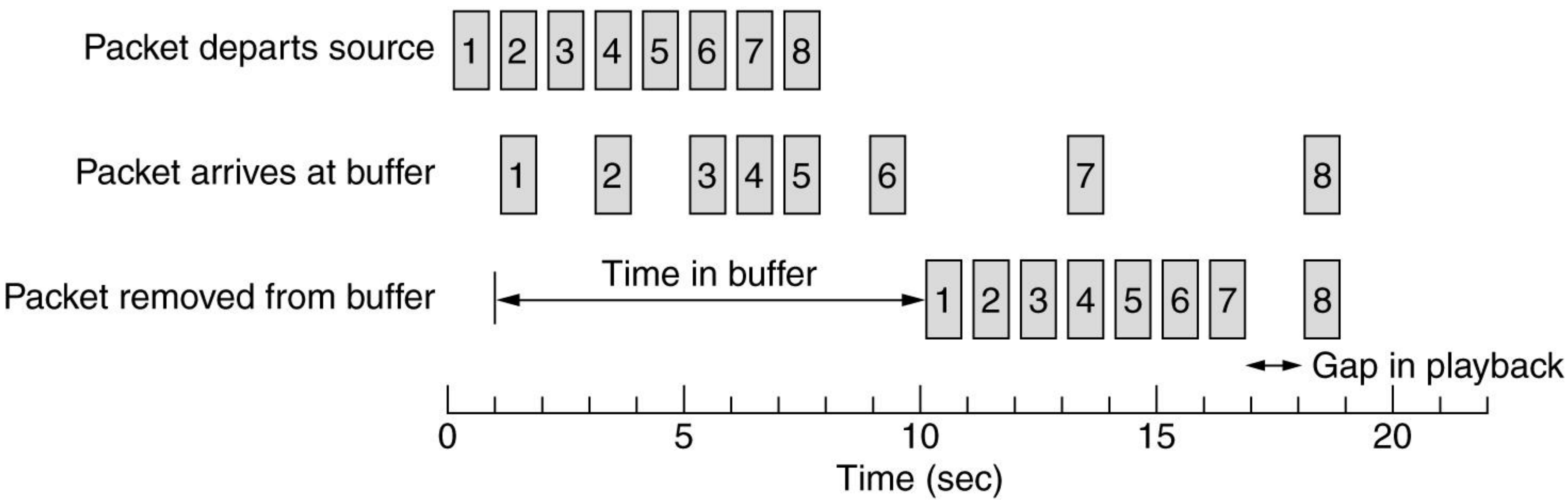
Requirements of QoS

A stream of packets from a source to destination is called flow. The needs of each flow can be characterized by four primary parameters: **reliability**, **delay**, **jitter** and **BW**. Together these determines the QoS the flow requires. Some common applications with required QoS is shown below.

Application	Reliability	Delay	Jitter	Bandwidth
E-mail	High	Low	Low	Low
File transfer	High	Low	Low	Medium
Web access	High	Medium	Low	Medium
Remote login	High	Medium	Medium	Low
Audio on demand	Low	Low	High	Medium
Video on demand	Low	Low	High	High
Telephony	Low	High	High	Low
Videoconferencing	Low	High	High	High

Buffering

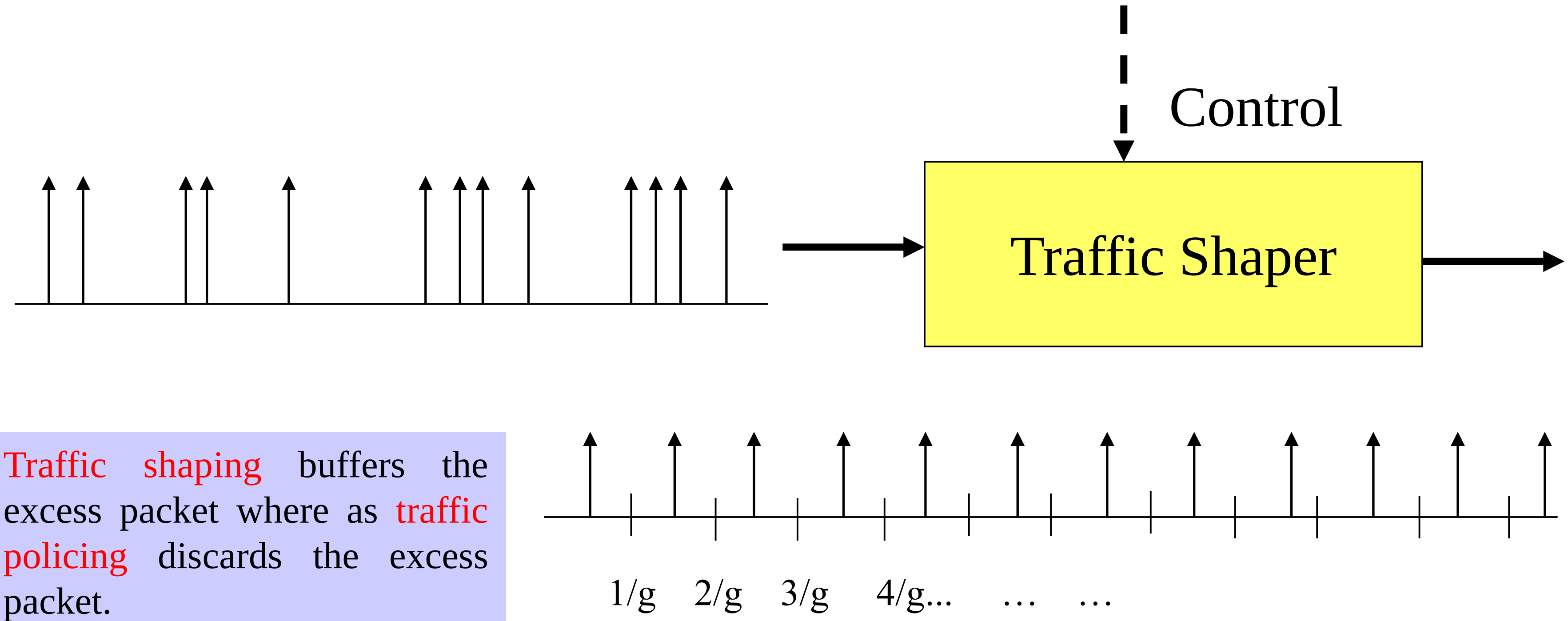
Flows can be buffered on the receiving side before being delivered. Buffering of traffic packet does not affect the reliability or BW; and increase the delay, but it smoothes out the jitter. For audio and video on demand, jitter is the main problem, so this technique helps a lot.



Smoothing the output stream by buffering packets.

Traffic Shaping

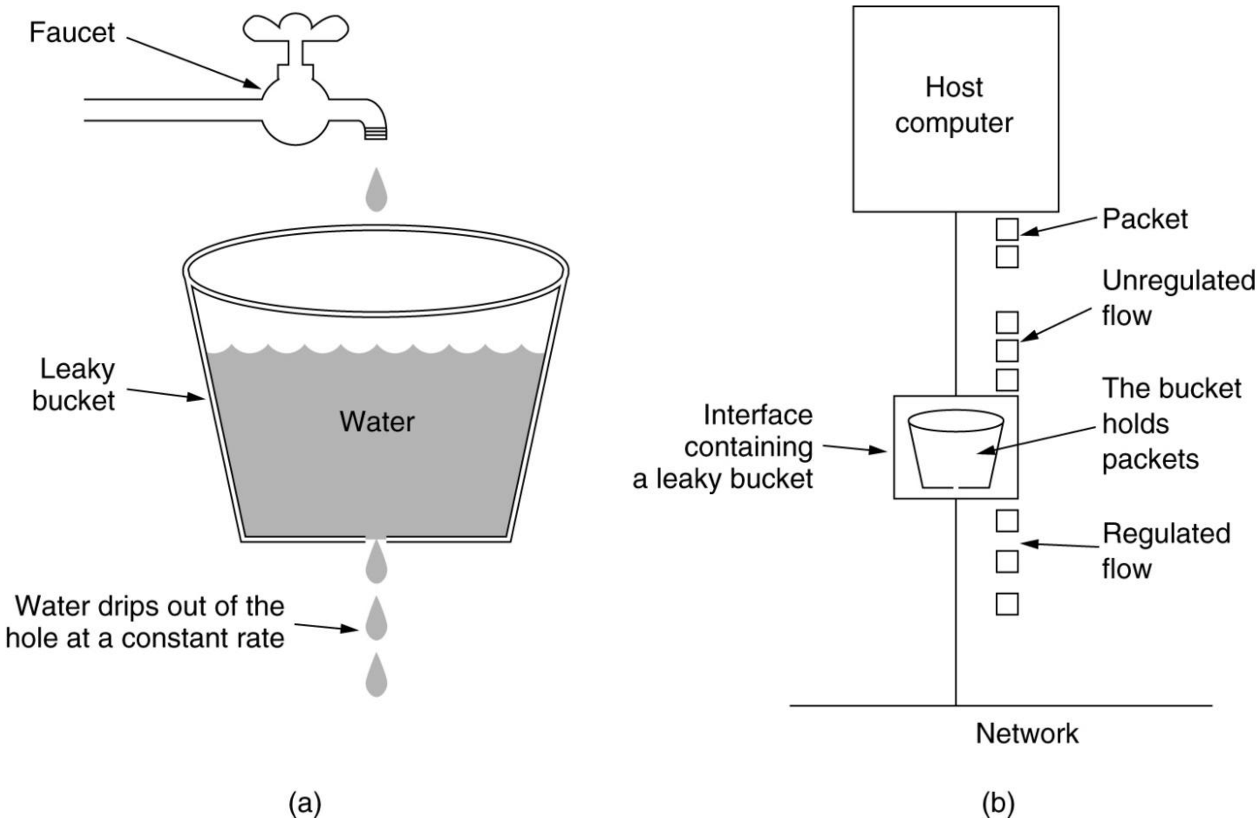
- ✓ Traffic shaping algorithm is used to convert turbulent incoming traffic to into a smooth, regular stream of packets.
- ✓ Two of most popular traffic-shaping algorithms are leaky bucket an token bucket.



Traffic shaping buffers the excess packet where as traffic policing discards the excess packet.

The Leaky Bucket Algorithm

Imagine a bucket with a small hole in the bottom, no matter the rate at which water enters the bucket, the outflow is at a constant rate (when there is any water in the bucket) and the flow is zero when the bucket is empty.



(a) A leaky bucket with water. (b) a leaky bucket with packets.

✓Once the bucket is full, any additional water entering it spills over the sides and is lost.

✓The same idea can be applied to packets. Conceptually each host is connected to network by an interface containing a leaky bucket i.e. a finite internal queue.

✓The host is allowed to put one packet per clock tick onto the network (the equivalent phenomenon of leaky bucket). Again this can be enforced by the interface card or by the operating system.

✓When the packets are all the same size (like ATM cells), this algorithm can be used as flow of cells/clock. However, when variable-sized packets are being used, it is often better to allow a fixed number of bytes per tick, rather than just one packet. Thus, if the rule is 1024 bytes/tick, a single 1024byte packet can be admitted on a tick, two 512 bytes packets, four 256byte packets, and so on

Example-1

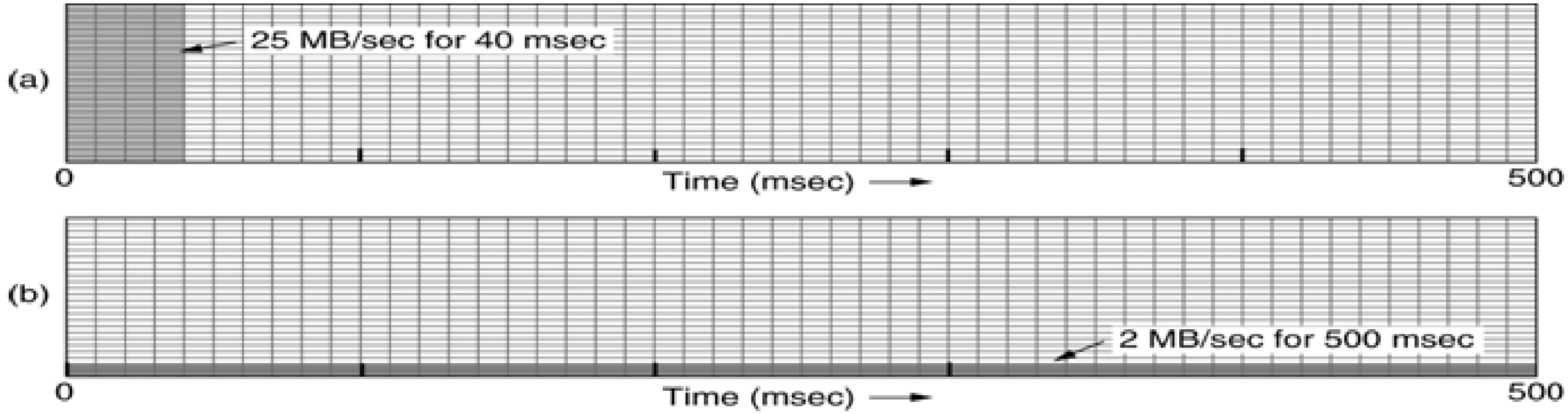
A node provides data burst at a rate R = 25 MB/sec for the duration of 40ms at the input of the leaky bucket. The output rate of the bucket is 2MB/sec. If the capacity of the buffer (or leaky bucket) is 10MB then determine (i) possible overflow (ii) time required to evacuate the bucket

Ans. (i) The amount of data burst enters the bucket is: 25×40/1000 (MB/sec)*sec = 1MB < Capacity of 10MB. Therefor no overflow of traffic.

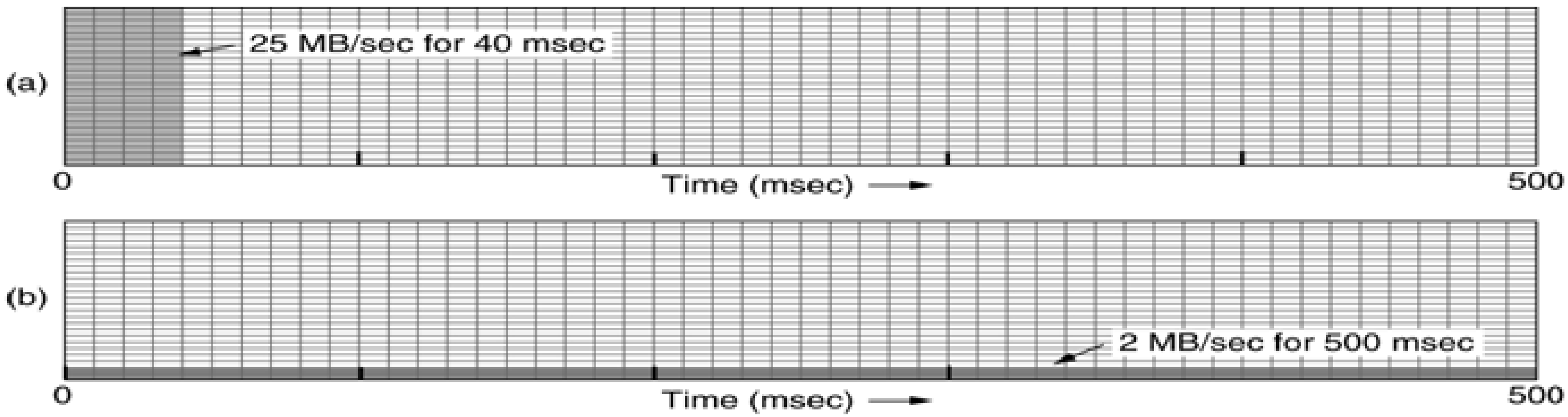
(ii) Let *t* be the time required to evacuate the bucket.

Now C= R*t or 1=2MB/sec**t* sce or *t* = 0.5 sec = 500 ms

The leaky bucket spread the burst over 500ms.



The input to the leaky bucket running at 25MB/sec for 40 ms. The output of the bucket draining out at a uniform rate of 2 MB/sec for 500ms.



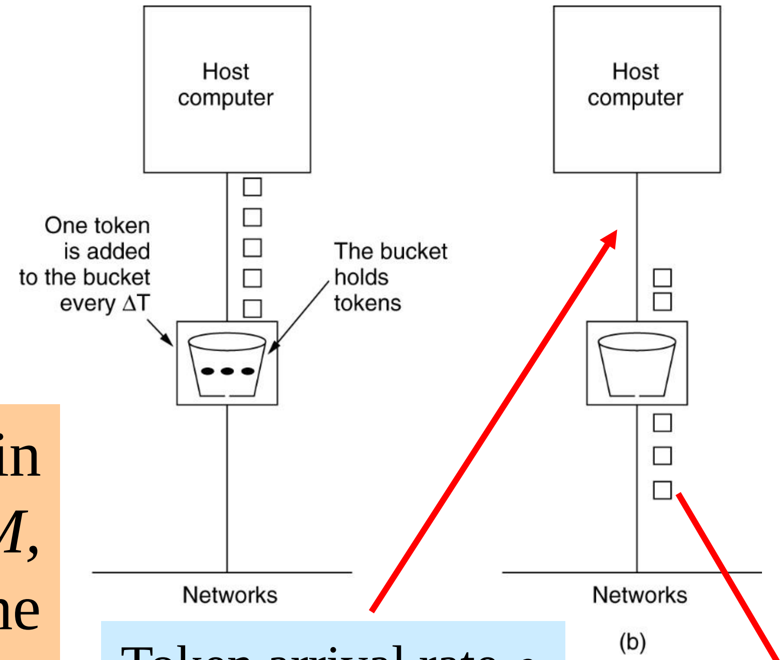
(a) Input to a leaky bucket. (b) Output from a leaky bucket

The Token Bucket Algorithm

✓The leaky bucket algorithm enforces a rigid output pattern at the average rate, no matter how bursty (unexpected huge traffic) the traffic is. For many application, it is better to allow the output to speed up somewhat when large burst arrive, so a more flexible algorithm is needed, preferably one that never loses data.

✓In this algorithm, the leaky bucket holds tokens, generated by a clock at the rate of ρ i.e. one token every ΔT sec.

✓The token bucket has two drain rate (i) maximum drain rate *M*, when token is reserved inside the bucket and (ii) low rate or taken rate ρ when no token is reserved inside the bucket



Token arrival rate ρ

Highest drain rate *M* MB/sec

(a) Before. (b) After.

✓Two parameters- the *token arrival rate* and the *bucket capacity* (Amount of token can be reserved) together describe the operation of the token bucket.

Consider a token bucket where token arrives at a rate of ρ= 2MB/sec. Assuming the token bucket is full when the 1MB burst arrives, the bucket can drain at the full rate M = 25MB/sec against reserved token of C = 250 KB.

During the draining of packet at 25MB/sec on the reserved token of C = 250 KB, the bucket will get some additional token which also be used to drain at a rate of ρ= 2MB/sec.

$M \rightarrow$ High speed drain rate
$\rho \rightarrow$ Low speed drain rate or token rate
$C \rightarrow$ Reserve token inside the bucket
$S \rightarrow$ Duration of high speed drain rate

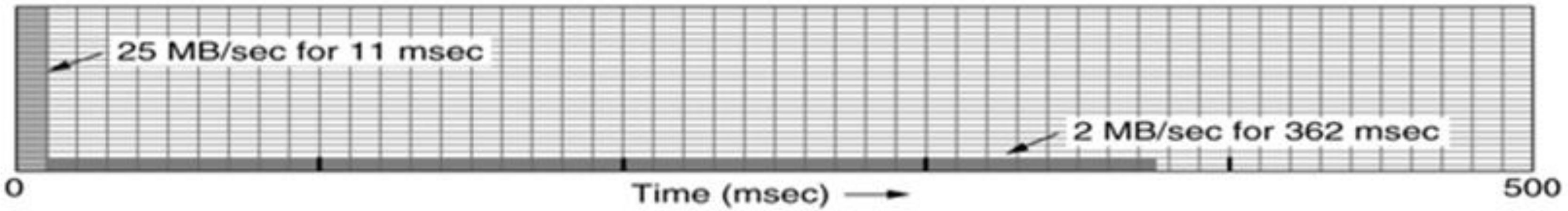
$$M*S = C + \rho*S$$

Total drained packet at the rate of M during S

Against the reserved token of C

Against the earned token during S

Calculating the length of the maximum rate burst is slightly tricky.
The capacity of the token bucket C = 250KB (Amount of reserved token)
Token arrival rate, ρ = 2MB/sec = 2000KB/sec
The maximum drain rate, M = 25MB/sec=25000KB/sec
Determine duration of both drain rate.



Let the duration of maximum rate is S.
Now, $M*S = C+ \rho*S$
Or, $S = C/(M- \rho)$ = 250/(25000-2000) sec = 250/23000sec
= 25/2300sec =10.86ms = 11ms
During 11ms only a fraction of the burst traffic will be released (11ms*25MB/sec = 275KB) at the rate M and the rest of the packet will be released at low rate ρ = 2MB/sec.
Here the volume of the burst is 1MB. Let the time required to drain the rest of the packet at a rate of ρ = 2MB/Sec is t.
1MB = (11/1000)*25+2*t
Or, t = 0.362sec = 362ms

Q. Consider the reserved token C = 500KB and M = 10MB/sec. Determine duration of both drain rate.

Ans. The capacity of the token bucket C = 500KB
Token arrival rate, ρ = 2MB/sec = 2000KB/sec
The maximum drain rate, M = 10MB/sec=10000KB/sec
Let the duration of maximum rate is S.
Now, $MS = C + \rho S$
$\Rightarrow S = C/(M-\rho)$=500/(10000-2000) sec=500/8000 sec = 5/80sec
= 62.5ms = 62ms
Here the volume of the burst is 1MB. Let the time required to maintain the token rate of 2MB/Sec to drain the entire burst is t.
1MB = (62/1000)*10+2*t
Or, t = 0.19sec = 190ms